

Element embeddings, distillation and evaluator performance in ACEpotentials: CrMnFeCoNi benchmarks and a Stage 2 proposal

ACEpotentials.jl — 14 September 2026 — accompanies EquivariantTensors PRs #144 and #145 and the ACEpotentials branches `fix/basis-ed-performance`, `pr/jax-port`, `pr/element-embeddings`

Summary

This report covers four things that came out of one line of work, and asks for decisions on a fifth.

Element embeddings. A linear ACE model whose species dependence enters through a **frozen element embedding** (Darby-style tensor reduction, channel folded into the radial index) was benchmarked against the standard **categorical** model on a five-component alloy, with labels distilled from MACE-MH-1. At matched polynomial degree the embedded model costs 7–17% on force RMSE for 3–6× fewer parameters; given one more degree — which its smaller design matrix makes affordable — it **matches** the categorical model at 2.4× fewer parameters (0.0767 vs 0.0763 eV/Å). The best absolute result is categorical degree 10 on 4 000 structures, 0.0641 eV/Å and 2.0 meV/atom, from a 151 GB design matrix. Learning curves show the categorical model is data-limited and the embedded model basis-limited, which is why the trade runs the way it does. The embedding’s **values** do not matter at $S = 5$ — which makes this a tensor reduction, not transferred chemistry, until $S > d_{\max}$.

Evaluator performance, and two fixes. The degree-6 student’s JAX GPU kernel runs 6× MACE-MH-1 at its fastest (MACE-torch + cuEquivariance, ≈ 1 100 atoms) and 25–100× at smaller cells; in LAMMPS the plugin path, not the model, is what costs a large ACE basis against MACE. The Julia evaluator ran the embedded model **slower** than the categorical one; profiling found both the design-matrix assembly (a type-unstable ForwardDiff Jacobian, 174–218× fixed) and the forward evaluation (LEN-wide radial spline tables of which only 6–8 columns are distinct, 9–12× fixed) — the same 151 GB assembly went from a projected 14 h to 25 min. The tensor-part Jacobian now lives in EquivariantTensors (#144), and a latent equivariance bug found on the way has its fix in #145.

Distillation and its levers. Bounding the structure generator halved the error at fixed basis; fixed $\sqrt{\rho}$ Finnis–Sinclair columns keep the fit convex and buy a few percent, the shape of the embedding function beyond sqrt is exhausted, and the species weights inside the density — learned in seconds by variable projection and transferable frozen — are what matters. This validates a two-stage route: pre-train the basis on a subset, then fit convexly at scale.

Prior art. Distillation into ACE and element embeddings are both published (Deringer’s group; Darby et al.; GRACE); what neither pacemaker nor gracemaker has is a tensor-reduced ACE fitted convexly, and EquivariantTensors #130 (CP/TRACE) is the learnable form of our frozen model.

Stage 2, for discussion. Whether the fitter stays in Julia or moves assembly to JAX is an open choice now that Julia assembly is fast; basis pre-training then convex fitting should become the default path, adopting #130’s mixing layer when it lands; and the convexity claim should be demonstrated against GRACE-FS on HEA25S — calibrated UQ (BLR and POPS), exact data selection, deterministic fits, delta-learning, hard constraints as QPs — gated by an accuracy-at-matched-cost experiment at $S = 25$.

Set-up

System and labels. Rattled and strained fcc CrMnFeCoNi cells (32–48 atoms, random equiatomic occupancy, no short-range order), generated with bounded uniform perturbations (lattice $\pm 3\%$, shear 0.02, rattle 0.02–0.10 Å, minimum separation 2.0 Å). Energies, forces and virials from MACE-MH-1 on an RTX A4500 (4 000 structures in 4.6 min). An earlier 6%-Gaussian generator produced crushed cells (36% compression, forces above 20 eV/Å) carrying 77% of the squared test error; bounding it halved the error at fixed basis and budget. MACE-MP-0-small was rejected as teacher: a difference-fit showed its disagreement with MH-1 to be rough, not a smooth offset.

Models. Order 3, ACE1 heuristics, uniform cutoff 6.25 Å. Categorical `ace1_model` with $S = 5$ elements; embedded `ace_embedding_model` with per-order widths $d_\nu = \min(d_{\max}, \binom{S+\nu-1}{\nu})$: **lossless** [5, 15, 35] (the full species-tensor width at every order; not a reparameterisation of the categorical model — it has strictly fewer functions, 475 against 1 348 at degree 6, because degree truncation treats the folded species index differently), $d_{\max} = 16$ ([5, 15, 16], order 3 cut to 16 of 35 directions) and $d_{\max} = 8$. Embedding table: MACE-MH-1’s 512-channel `node_embedding` reduced by PCA of the five rows (Gram preserved exactly; generic orthonormal mixing for $d > S$). Parameters are $n_B \times S$ (a readout per central species).

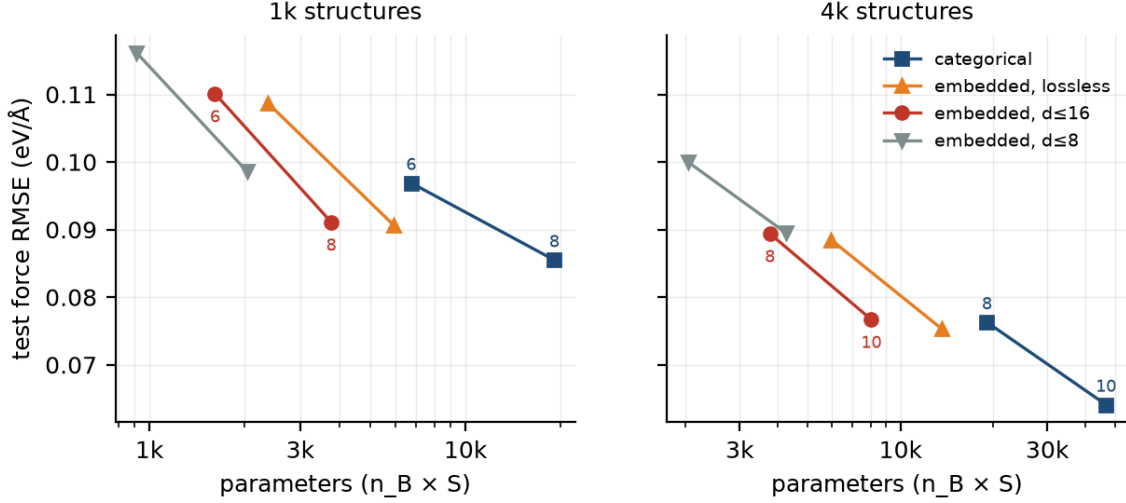


Figure 1: Test force RMSE against parameter count. Point labels are the polynomial degree. On the 4k set the degree-10 embedded $d \leq 16$ model (8 045 parameters) sits at the accuracy of degree-8 categorical (19 120).

Fitting. Held-out 80/20 split; algebraic smoothness prior $p = 4$; weights from `acefit!` defaults; factor-once Tikhonov (QR of A , SVD of R) with λ swept per model and the best held-out force RMSE reported. Assembly on 12 workers; the degree-10 categorical matrix ($404\,144 \times 46\,885$, 151 GB) was factorised in place on a 376 GB node.

Results

model	n_B	params	1k test F	4k test F	4k test E	rel. to cat.
deg 6 categorical	1 348	6 740	0.0969	—	—	—
deg 6 embedded $d \leq 16$	323	1 615	0.1101	—	—	+14%, 4.2× fewer (1k)
deg 8 categorical	3 824	19 120	0.0855	0.0763	0.0024	—
deg 8 embedded lossless	1 190	5 950	0.0906	0.0885	0.0025	+16%, 3.2× fewer
deg 8 embedded $d \leq 16$	753	3 765	0.0911	0.0894	0.0025	+17%, 5.1× fewer
deg 8 embedded $d \leq 8$	408	2 040	0.0986	0.1000	0.0027	+31%, 9.4× fewer
deg 10 categorical	9 327	46 635	—	0.0641	0.0020	—
deg 10 embedded lossless	2 730	13 650	—	0.0753	0.0022	+17%, 3.4× fewer
deg 10 embedded $d \leq 16$	1 609	8 045	—	0.0767	0.0022	+20%, 5.8× fewer
deg 10 embedded $d \leq 8$	850	4 250	—	0.0895	0.0025	+40%, 11× fewer

Table 1: Held-out force RMSE (eV/Å) and energy RMSE (eV/atom) against MH-1 labels; 1k = 800/200 split, 4k = 3 200/800. “rel. to cat.” compares with the categorical model at the same degree on 4k.

Three things follow from Table 1 and Figure 1. First, at matched degree the embedding costs 14–20% on forces (energies are unaffected: 2.2–2.7 meV/atom throughout), for 3–6× fewer parameters. Second, **lossless and $d \leq 16$ are indistinguishable at every degree** — truncating order 3 from 35 to 16 species directions costs nothing measurable at $S = 5$ — whereas $d \leq 8$, which also truncates order 2, costs about one degree. Third, per basis function the embedded model wins at every point: degree-10 $d \leq 16$ matches degree-8 categorical at 2.4× fewer parameters, and it was affordable (26 GB) where the categorical alternative needed a 376 GB node.

The learning curves (Figure 2) explain the table. Both degree-6 models saturate — train and test meet — by 400 structures; the embedded model gets there by 200 and is the better model below that. At degree 8 the categorical model still has slope at 800 and duly gains 11% from 4× the data, while the embedded models gain 2%. **The embedded basis wants degree, not data**; the categorical basis wants data. This is the mechanism behind “accurate per structure vs accurate per parameter”.

The embedding’s values do not matter at $S = 5$

A 2×2 of embedding source (MP-0-small, MH-1) $\times$ reduction (first- d column truncation, PCA) at degree 6 gave test F within 0.001 eV/Å at $d \leq 16$ and 0.005 at $d \leq 8$, although the truncated tables’ element-similarity structure correlates only 0.5–0.7 with the full tables’. At $S = 5$ the frozen embedding acts as a generic tensor reduction, consistent with Darby et al.’s random-projection result — and with the tables themselves: the full 89×128 MP-0 table has rank 89 with a slow, featureless singular-value decay (64 components for 98% of the energy), i.e. no low-rank “chemical” structure to exploit; transferred chemistry can only show once $d_{\max} < S$ truncates order 1 itself. PCA remains the

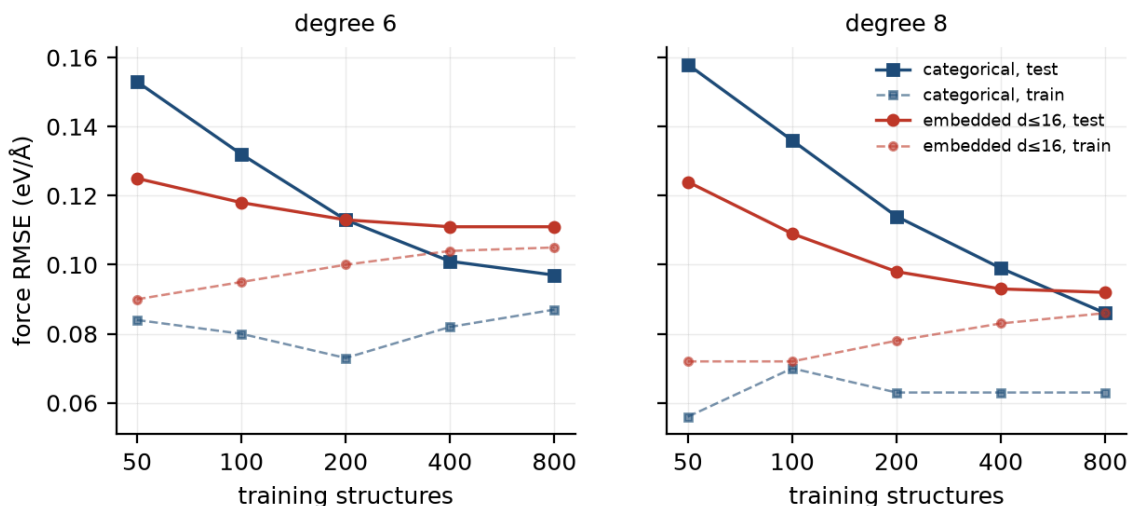


Figure 2: Learning curves on the 1k set (test solid, train dashed). Degree 6 is basis-limited for both models; at degree 8 the categorical model is still data-limited (gap 0.022 at $N = 800$) while the embedded one has converged.

right construction (it was also what exposed that a DCT mixing frame spans only 9/15 and 13/35 of the species tensors — a genericity check now in the tests).

Evaluation cost

The JAX evaluator’s GPU kernel runs 32–38× MACE-MH-1’s in f32 at 40–1 300 atoms (6–11× end-to-end with the host neighbour list) and $7.4\text{--}8.3 \times 10^4$ atom-steps/s in f64 — about 100× MH-1’s f64 rate of $7.6\text{--}8.6 \times 10^2$, because the ACE kernel is gather-bound and barely notices the card’s 1/64 fp64 rate while MACE’s dense matmuls do; the same holds on the A4500 (4.8×10^4 f64 at 40 atoms). MH-1 does not fit a 3 000-atom cell on 20 GB in either precision.

The MACE numbers above are the JAX stack without fused kernels, and that stack cannot be accelerated as shipped. Checked directly: the mace-jax bundles run plain e3nn-jax; enabling the cueq backend routes through cuequivariance_jax, but every mace-jax adapter except one calls it with method='naive' — the pure-JAX reference implementation — so the HLO contains no custom kernels and the “accelerated” model is 2–20% **slower**, at 4×10^{-9} eV/Å agreement. So the fair question is what MACE does at its best, and that was measured in a one-off, separate MACE-torch environment (torch 2.14, mace 0.3.16, cuEquivariance-torch 0.11, same A4500, same frames; MH-1 torch vs mace-jax agree to 1 meV / 3×10^{-4} eV/Å):

Three things follow (Figure 4). cuEquivariance is a large effect at scale — 6.4× on MH-1 and 7.2× on MP-0 medium at ≈ 100 atoms, where torch + cuEquivariance is 1.8× and 2.6× the mace-jax stack — and none at all below ≈ 100 atoms, where torch dispatch dominates and mace-jax is the fastest MACE (5.3×10^3 vs 1.0×10^3 on MP-0 medium). The best MACE-MH-1 measured is therefore torch + cuEquivariance at large cells, 1.4×10^4 atom-steps/s, still rising with size; against it the student’s kernel speed-up is 6× at ≈ 100 atoms (109× at 40 atoms, 25× at 330), not the 11–38× quoted

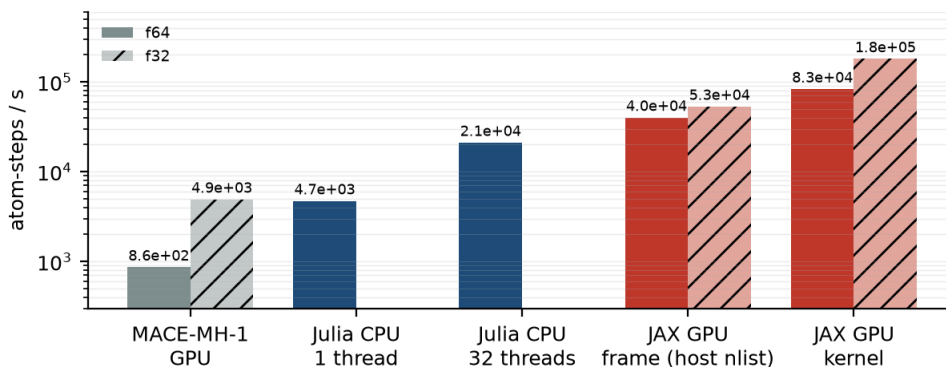


Figure 3: Throughput of the degree-6 $d \leq 16$ student on one host (lestrade: 32 cores, RTX 4000 Ada), 256–384-atom cells, f64 (solid) and f32 (hatched). The JAX kernel runs 32–38× the teacher in f32 and about 100× in f64 **through mace-jax**; against MACE-torch + cuEquivariance at large cells the f32 figure is 6× (Figure 4). A single Julia core matches the teacher on a GPU. Julia is f64 only. Neither card is a production GPU: fp64 runs at 1/64 of fp32 on both the RTX 4000 Ada and the A4500. The f32 bars use XLA’s defaults, i.e. TF32 matmuls on these cards (a 1.2×10^{-3} descriptor error unless `jax_default_matmul_precision=highest` is set, which acejax’s production entry points do); the f64 bars are exact.

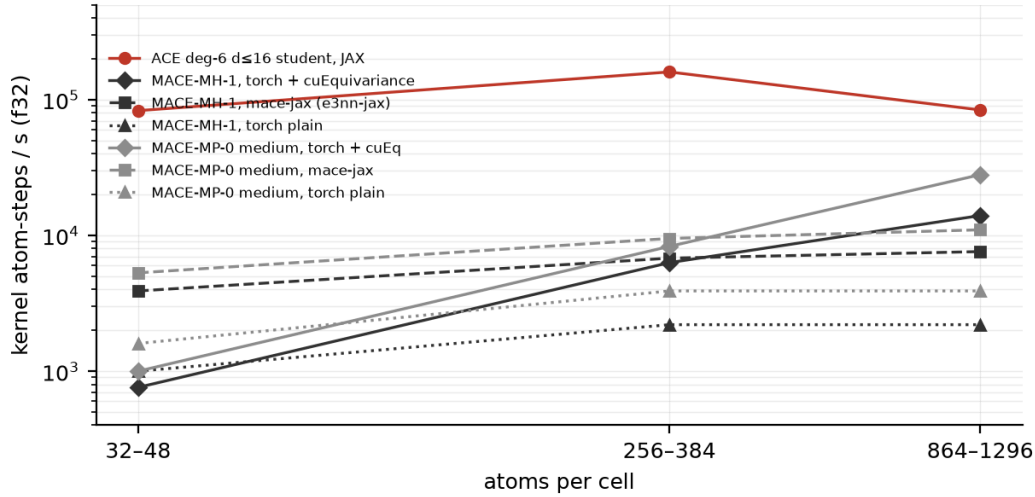


Figure 4: Kernel throughput in f32 on the A4500 against cell size: the degree-6 embedded student (JAX) and MACE-MH-1 / MACE-MP-0 through three stacks — MACE-torch with cuEquivariance, MACE-torch plain, and mace-jax (e3nn-jax). MACE-MP-0 medium ($L=1$) through all three stacks. cuEquivariance is worth $1.3\times \rightarrow 6.4\times$ to torch as the cell grows, and torch + cuEquivariance overtakes mace-jax above a few hundred atoms.

against mace-jax, and the end-to-end (ASE-frame) ratio there is $3\times$. That is the number to carry into any comparison with a production MACE deployment on a larger card, where cuEquivariance’s advantage would grow further. It does not change the accuracy results or the Julia-vs-JAX comparison.

The Julia evaluator, however, ran the **embedded** model $1.8\times$ slower than the categorical one despite $4\times$ fewer basis functions, and both far below a well-written CPU kernel. Profiling found the cause: the splined radial tables are LEN wide (224 columns per edge for the embedded model), yet because the embedding weights make every R_{nl} a scalar multiple of $P_{n'}$, only 6–8 columns are distinct — the evaluator computes them all, as `SVector{LEN, Dual}`. A prototype exploiting the factorisation ($R_{nl} = c_{n,\text{pair}} P_{n'}$, the structure acejax already uses) with an allocation-free site loop is exact to 10^{-12} and $12\text{--}18\times$ faster on four cores, above the JAX fp64 GPU kernel. On the JAX side the one known size-dependent cost — the reverse-mode scatter behind the edge gather, which degraded the Apple-Silicon MD path $2.3\times$ between 216 and 1 728 atoms — has a measured cure (an algebraically identical one-hot matmul, $5.1\times$ end-to-end with bit-identical trajectories) that is in acejax as `edge_a_kind="matmul"` but has not yet been calibrated on the A4500, where the kernel drops $2\times$ between ≈ 330 and $\approx 1\,100$ atoms (Phase 15).

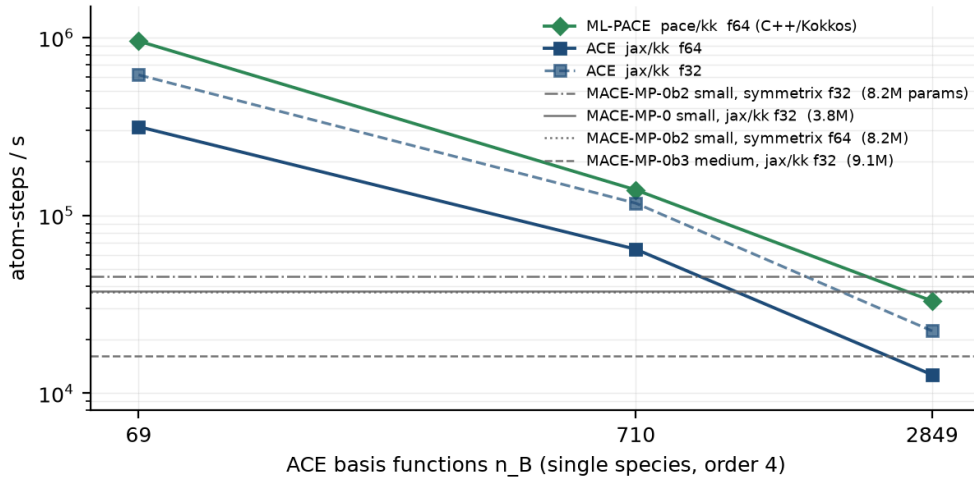


Figure 5: LAMMPS throughput at 1 728 Si atoms, one rank, one RTX A4500 (moriarty). ACE through `pair_style jax/kk` at three basis sizes (order 4, $l_{\text{max}} 4\text{--}5$, cutoff 6 Å), ML-PACE pace/kk at matched basis shape (78 / 693 / 2 874 functions, 0.9–13% off ours), and three MACE foundation models through `jax/kk` (lammps-jax) or `symmetrix` (hand-written Kokkos). Every engine was gated against the others before timing: ACE `jax/kk` vs Python $1\text{e-}13$ eV/Å; `symmetrix` vs `jax/kk` on the same MACE checkpoint $4\text{e-}5$ eV/Å.

In LAMMPS: against ML-PACE and MACE

Figure 5 puts the JAX evaluator in its production setting. Against **ML-PACE** at matched basis, `jax/kk` is 2.1–3.0× slower in f64 and 1.2–1.5× in f32, roughly flat across basis size; the gap is not in the kernel — in Python the same model matches pace at matched basis — but in the plugin path (padding to a fixed edge capacity, ghost atoms, the `comm` mode; measured 3× at the large basis, 1.05× at the small). Against **MACE**, the 69- and 710-function ACE models run 16× and 3× MACE-MP-0 small through the **same** pair style; the 2 849-function model runs at 0.6×. **That last figure is the plugin path, not the model:** called from Python on the same card the same 2 849-function ACE does 5.5×10^4 atom-steps/s in f32 — 1.5× MACE-in-LAMMPS — and `jax/kk` retains only 0.41 of it (0.34 in f64), against 0.55–0.63 for the two smaller bases. The loss is concentrated where the per-atom stages dominate (A2B 54%, AA 38% of `site_basis` at this shape) because our bundle pads the atom axis 3.7× (locals + ghosts + margin) and evaluates those stages over every slot, whereas the `lammps-jax` MACE bundle runs in `comm` mode with `--owned-rows` and does not pay for ghosts. Two further contributions are card-scale rather than harness: a gather-bound kernel is bandwidth-limited on a 20 GB A4500 where MACE’s dense 128-channel matmuls are what the card does best, and the reverse-mode scatter behind the edge gather (Phase 15) is uncalibrated here. The fixes are known and cheap: `owned-rows` semantics for the ACE bundle, the Phase 15 calibration, and a MACE-in-Python point so both sides carry a retention figure. Two consequences for this report: the 5-element embedded models above sit at 323–1 609 functions, i.e. between the middle and large points here, so their LAMMPS throughput is expected in the $2\text{--}6 \times 10^4$ range rather than the 10^5 the isolated kernel reaches; and the same `jax/kk` engine runs MACE-MP-0b3 medium 1.4× **faster** than `symmetrix` does, so the JAX export path is not itself the handicap.

Stage 2 proposal, for discussion

The plan (`docs/plans/jax_ace_port_plan.md`, Stage 2) has been revised against these results. Its shape, and the decisions it asks for:

2A — linear fit at scale: an open choice. Two routes. (i) **Stay in Julia:** assembly and solve where the model, the priors, the parameter-gradient rrules and the data path already live — a single source of truth — with **phase 18** (row-block assembly feeding TSQR, which still yields R for BLR, or ScaLAPACK on the same operator per the `gap_fit` recipe; LSQR beyond the R -factor floor) for what no single node holds. (ii) **Port assembly to JAX:** the design matrix built on the GPU through the same differentiable descriptor the evaluator uses, so fitter and evaluator cannot diverge, with the solve in whichever runtime holds A . The case for (ii) was strongest when Julia assembly was 100× slower than its arithmetic warranted; the performance branch removed that premise — the 151 GB degree-10 matrix now assembles in 25 min on CPU, and basis pre-training (2B) needs nothing Julia lacks. What remains for (ii) is real but smaller: GPU bandwidth ($\approx 10\times$ on a gather-bound assembly, not $100\times$), the shared descriptor, and MACE’s own move to JAX. Sizing either way: at 4 000 structures the categorical degree-12 matrix is 330 GB and needs phase 18 on any route; the embedded one is 50 GB. The phase-18 spike settled the solver: ScaLAPACK.jl is dead (Julia-0.4 code) but the library works from SCALAPACK32_jll + MPI.jl; a 26-line TSQR over MPI.jl is the best direct method on ACEfit’s row-block layout (agreement with LAPACK to $1e\text{--}12$ at cond $1e21$, $13\times$ faster than ScaLAPACK on that layout), with the $n \times n$ R factor as its memory floor (162 GB at embedded degree 20); LSQR’s damp is QR’s λ for $\lambda \geq 10^{-3}$ and is the only route beyond that floor; GPU QR (JAX/lineax) is exact but only $1\text{--}2\times$ a 32-thread CPU at the sizes that fit 20 GB and OOMs at the degree-10 shape. A third route — compiling the Julia model to XLA with Reactant, which would make the language question moot — remains blocked: the standard ETACE path traces on a CUDA host except for composite (struct-of-array) inputs to KernelAbstractions kernels (EnzymeAD/Reactant.jl#3267), a data-layout change in EquivariantTensors rather than a compiler fix. Rust and PyTorch were assessed and set aside; the plan records the reasoning. *Decision sought: (i) or (ii) — or (i) now with (ii) held open, using the $S = 25$ gate experiment (a ≈ 150 GB assembly) as the test of whether the Julia route suffices at the scale that matters.*

2B — basis pre-training, then convex fit. Everything non-linear (radial mixing W_{nlq} , the element embedding E , Finnis–Sinclair densities as linear combinations of the order-1 basis) is learned once on a subset by Kaufman variable projection — the gradient at fixed c^* , which the existing `grad_params` / `pullback_2_mixed` / `energy_forces_virial` rrules already provide in Julia — then frozen, and the coefficients are fitted convexly at scale. The spike passed its gate: learned densities beat hand-chosen ones with fewer columns and transfer frozen to a disjoint split. This is the GRACE-FS / PACE function class split into two stages; `pacemaker`, `gracemaker` and MACE all learn their radials, and ACEpotentials’ fixed heuristic radials are the outlier. *Decision sought: make pre-trained radials and densities the default path rather than an option; two Julia items to build (a structured $W_{nlq}(E)$ and the FS term at the hook already marked in `ace.jl`).*

2C — the convexity demonstration against GRACE-FS. Neither `pacemaker` (linear, categorical) nor `gracemaker` (embedded, gradient-descent) fits a tensor-reduced ACE convexly; that is the differentiator, and it has to be shown on

the data the competition used – HEA25S, teacher GRACE-2L-OMAT (public), their extended-distillation recipe. Five measurable benefits: calibrated UQ for free (BLR, and Swinburne–Perez POPS for the misspecified regime a distilled student is always in) against a GRACE-FS ensemble; exact D-optimal data selection (error vs N , selected vs random); one-shot deterministic fits with closed-form λ ; delta-learning on DFT as one more linear solve against GRACE’s fine-tuning curve; and hard physical constraints as convex QPs (equilibrium, elastic constants, phase ordering, repulsion). *Gate first*: accuracy at matched MD cost at $S = 25$ on a 5 000-structure subset (≈ 150 GB, fits the 376 GB node today) – the first regime where $d_{\max} < S$ and the embedding’s values can matter. If the linear model does not reach GRACE-FS accuracy there, items 1–5 are wins on a worse model. *Decision sought*: approve the gate experiment as the next substantial run, ahead of phase 18.

Upstream items. A latent correctness bug in *EquivariantTensors*, found by the embedding spike – now *EquivariantTensors* #145, in review: `sparse_equivariant_tensor` (the singular form, which `ACEpotentials.Models._generate_ace_model` calls) silently returns a **non-equivariant** basis when `mb_spec` is not grouped by correlation order – `SparseSymmProd` re-sorts the spec but the `A2Bmaps` columns are not permuted to match; measured rotation error 0.04–0.86 for ungrouped specs against 5×10^{-16} grouped. *ACEpotentials* is safe today only because its spec generator happens to emit order-grouped specs. The plural `sparse_equivariant_tensors` already guards against this; the fix with a regression test is #145. The row-wise pushforwards are #144 (`pushforward_rows!` for `PooledSparseProduct`, `SparseSymmProd` and `SparseACEbasis`, 0.5.2), in review. Two further items: ET #130 (CP/TRACE format, on restructure) is the learnable form of our frozen embedding – `EquivLinearL + CPACEbasis` with `W` frozen is this model, with a per-`l` `W` more general than our species-only one – so Stage 2B’s learnable mixing should be adopted from it when restructure lands rather than built in *ACEpotentials*, and its deferred efficient Jacobian is `pushforward_rows!` looped over rank (a natural follow-up to #144); ET #83 (`NeighbourLists` 0.6, multithreaded and GPU neighbour lists) is being rebased now that 0.6.2 is released; `SparseSymmProdDAG` for the CPU evaluator; *ACEfit*’s per-task serialisation, `GC.gc()` and `Array(A)`.

The pull requests

branch (PR)	based on	contents
<i>EquivariantTensors</i> #144 (review)	ET main 0.5.1	<code>pushforward_rows!</code> – row-wise (per-neighbour) vector-tangent pushforwards for <code>PooledSparseProduct</code> , <code>SparseSymmProd</code> , <code>SparseACEbasis</code> ; 0.5.2; ET suite 5 916/5 916
<i>EquivariantTensors</i> #145 (review)	ET main 0.5.1	canonical <code>Aspec</code> ordering in <code>sparse_equivariant_tensor</code> (the singular constructor silently returned a non-equivariant basis for ungrouped specs); regression test; independent of #144
<code>fix/basis-ed-performance</code>	main + ET #144	assembly and forward-evaluation speed-ups; profiling findings; 11 commits; the tensor-part Jacobian is ET’s <code>pushforward_rows!</code> , <code>compat EquivariantTensors = "0.4.3, 0.5"</code>
<code>pr/jax-port</code>	<code>fix/basis-ed-performance</code>	JAX evaluator port: <code>acejax</code> , exporter, tests + CI divergence gate, LAMMPS, benchmarks, plan; 7 commits, no changes under <code>src/</code>
<code>pr/element-embeddings</code>	<code>pr/jax-port</code>	frozen element embeddings, factorised radial export, distillation benchmarks, FS/VarPro spikes, this report; 5 commits

Table 2: The pull requests. The two *EquivariantTensors* PRs are open for review; the three *ACEpotentials* branches are local and stacked, and the first depends on the release of #144 as 0.5.2. Fine-grained history (146 commits) is on branch `jax-eval`; the PR branches are squashed into logical commits.

Recommendations

1. **Use the embedded model where parameters, memory or many elements are the constraint, and give it one more degree than the categorical model would get.** Degree-10 $d \leq 16$ is the current best trade (0.0767 eV/Å, 8 045 parameters, 26 GB matrix on 4k). Where accuracy per structure is all that matters and a large node is available, categorical degree 10 (0.0641) still wins; categorical degree 12 (330 GB) needs the distributed assembly of Stage 2 phase 18, embedded degree 12 (≈ 50 GB) does not.
2. **Keep $d_{\max} = 16$; do not use $d_{\max} = 8$** (it truncates order 2 and costs a full degree).
3. **Treat the embedding source as immaterial at $S \leq d_{\max}$** , and test the transferred-chemistry claim only at $S = 10$ –25, where the GRACE-FS distillation on HEA25 is the comparison to beat.
4. **Code changes, as three stacked pull requests against main** (listed in Table 2, with the two *EquivariantTensors* PRs they build on). (a) `fix/basis-ed-performance`: the design-matrix path – `evaluate_basis_ed` replaced by a type-stable pushforward Jacobian and the boxed accumulation in `energy_forces_virial_basis` removed – 174–218× per structure, exact to 10^{-12} ; and the forward path – factorised radial spline tables with a plain-Float64 kernel, an allocation-free site loop with $w_{AA} = A2B^T W_B$ folding, neighbour-list reuse – 9–12× on four threads, exact to 10^{-12} ; 1 300 existing tests plus 471 new ones pass. The 151 GB degree-10 assembly above took 25 min

on it instead of a projected 14 h. Its tensor-part pushforward is EquivariantTensors #144 (pushforward_rows!, in review), on which the ACEpotentials PR depends — the branch already calls it, and its three interim kernels are gone; the ACEfit follow-ups (per-task model serialisation, GC.gc() per task, the Array(A) copy) are listed in the PR text. (b) pr/jax-port: the JAX evaluator, exporter, LAMMPS bundle, benchmarks and the CI matrix that re-exports fitted models from the ACEpotentials under test and requires the evaluator to reproduce them. (c) pr/element-embeddings: everything in this report — ace_embedding_model, the PCA reduction with its rank check, the factorised radial export, the distillation pipeline, the spikes and the findings.

5. **After the radial fix, revisit the recursive (DAG) AA products for the CPU evaluator.** The Phase-8 spike (acejax/spike_recursive/) measured subproduct sharing at **0.99×** on energy+forces for the JAX GPU evaluator in f64 (1.19× in f32; the reverse pass must materialise $\approx 25\,000$ intermediate columns, and AA is only 18% of site_basis on the GPU once measured against an oracle rather than in isolation — so it is a dead end there), but **2.30×** on CPU, where AA is 75% of the energy+force cost. With the radial splines no longer dominant in Julia, the AA products become its largest stage, which is the CPU regime the spike measured; EquivariantTensors already provides SparseSymmProdDAG, so the trial is a swap of the tensor’s aabasis on the fixed kernels.
6. **Retire the design-matrix cache** (it exhausted the storage quota at 200 GB; assembly is now minutes) and **replace svd(R)** in the factor-once solver with a Cholesky per λ or a threaded SVD — at $n = 46685$ it is the remaining post-assembly cost.
7. **Generator before basis.** Bounding the perturbations halved the error; adding short-range order and the unary/binary structures of GRACE’s “extended distillation” are the next data-side steps and cost minutes of GPU.
8. **Next accuracy lever, convexly:** fixed $\sqrt{\rho}$ Finnis–Sinclair columns with learned (VarPro) species weights inside the density gave -12% at degree 4 and transfer frozen to a disjoint split; the shape of the embedding function beyond $\sqrt{\cdot}$ does not matter. This is the like-for-like answer to GRACE-FS and should be included in any $S > 5$ run.

Sources: acejax/bench/distil/RESULTS.md; docs/findings/FINDINGS_assembly_profile.md, FINDINGS_forward_profile.md, FINDINGS_fs_spike.md, FINDINGS_varpro_spike.md, FINDINGS_codes_survey.md; plan docs/plans/jax_ace_port_plan.md (Stage 1D–E, 2B–2C).